The Babylon Continuation: Deterministic Replay and Refusal-First Execution in Language-Driven Systems

J. M. Bookbinder
Witness Grade Analytics (WGA)
Atlanta, Georgia, USA

March 2026

Abstract

All claims in this paper are contingent on artifact verification. No claim is authoritative without a corresponding run bundle and decision receipt.

Modern systems increasingly rely on language as an execution surface for initiating, modifying, and validating process state. This paper introduces a deterministic continuation control framework for language-driven systems, enforcing a decision model with precedence HALT > INCOMPLETE > PASS. The method requires replay-verifiable artifacts, cryptographic integrity checks, and explicit refusal records when continuation cannot be justified. Language is treated as telemetry rather than narrative, and execution is permitted only when structural admissibility is preserved under the declared evaluation scope, with deterministic replay required where verified.

1 Introduction

Language has become an operational interface for modern systems, including software pipelines, financial systems, and artificial intelligence workflows. However, current systems lack a deterministic method for verifying whether execution should continue once language has altered system state. This paper presents a continuation control architecture in which language is treated as structured signal input, and system execution is governed by replay-verifiable evidence rather than narrative interpretation [2].

2 Continuation Decision Model

The system evaluates continuation under strict precedence:

HALT > INCOMPLETE > PASS

- HALT: Structural violation or missing invariant prevents continuation.
- INCOMPLETE: Required evidence or artifacts are missing.
- PASS: All constraints satisfied within the declared evaluation scope.

3 Deterministic Replay Requirement

Admissible execution under replay requires reproducibility under identical inputs. This condition is enforced when replay certification is within the declared evaluation scope. The system enforces:

- Artifact hashing (SHA-256) [4]
- Run bundle integrity
- Replay equivalence

Any mismatch between original and replayed execution results in immediate HALT.

4 Refusal-First Execution Control

If continuation cannot be justified, the system must emit a refusal record rather than proceed.

- No silent failure
- No narrative override
- Explicit obstruction documentation

Refusal is treated as a valid and required system outcome.

5 Language as Telemetry

Language is not treated as explanation but as a structured signal capable of altering system state. All language inputs must be:

- Captured as artifacts
- Hash-bound
- Replay-tested

This prevents semantic drift and ensures that meaning is tied to verifiable structure.

6 Cross-System Language Admissibility Experiment (Claude vs Gemini)

6.1 Objective

This experiment evaluates whether contemporary large language model systems preserve structural admissibility under controlled input conditions. Specifically, we test whether two independent systems, Claude and Gemini, produce outputs that satisfy:

1. **Instructional Fidelity**: preservation of explicit constraints
2. **Operator Binding**: continuity between input instruction and output action

3. **Replay Consistency**: stability of output under repeated identical inputs
4. **Structural Preservation**: retention of formal decision logic HALT > INCOMPLETE > PASS

The experiment determines whether outputs qualify as DIRECT witnesses, or degrade into WRAPPED or DRIFT classes under the continuation control framework.

6.2 Experimental Setup

Systems Evaluated

- System A: Claude (Anthropic) [1]
- System B: Gemini (Google) [3]

Input Construction
Each system was provided with identical structured prompts containing:

- the decision model HALT > INCOMPLETE > PASS
- a deterministic reasoning requirement
- an artifact-first output requirement
- a constraint against narrative substitution

Execution Protocol

1. Submit prompt P
2. Record output O_i
3. Repeat execution for $n \geq 2$
4. Evaluate replay stability and constraint preservation

6.3 Evaluation Criteria

Each output is evaluated against four gates:

- **G1 — Constraint Preservation**
- **G2 — Decision Integrity**
- **G3 — Replay Determinism**
- **G4 — Operator Continuity**

Property	Claude	Gemini
Constraint Preservation	Partial	Low
Decision Integrity	Preserved	Inconsistent
Replay Determinism	Incomplete	Failed
Operator Continuity	Partial	Weak

Table 1: Observed system behavior under controlled prompt conditions.

6.4 Results

6.5 Witness Classification

Witness classes are defined as:

- **DIRECT**: full structural preservation and deterministic replay
- **WRAPPED**: partial structural preservation with transformation
- **DRIFT**: structural degradation or re-authoring

Observed Classification

- Claude → WRAPPED
- Gemini → DRIFT

No system produced a DIRECT witness.

6.6 Decision Outcome

```
{
  "decision": "HALT",
  "reason": "no_system_produced_direct_witness",
  "precedence": "HALT > INCOMPLETE > PASS"
}
```

6.7 Interpretation

Both systems exhibit transformation behavior of the form

$$O = f_{\mathrm{LLM}}(P)$$

where f_{LLM} is non-deterministic and does not guarantee constraint preservation.

Thus, large language model systems function as probabilistic transformation operators rather than deterministic execution systems.

6.8 Implications for Continuation Control

Because neither system satisfies admissibility requirements:

- outputs cannot be accepted without verification
- systems must be treated as untrusted generators
- admissibility must be enforced externally

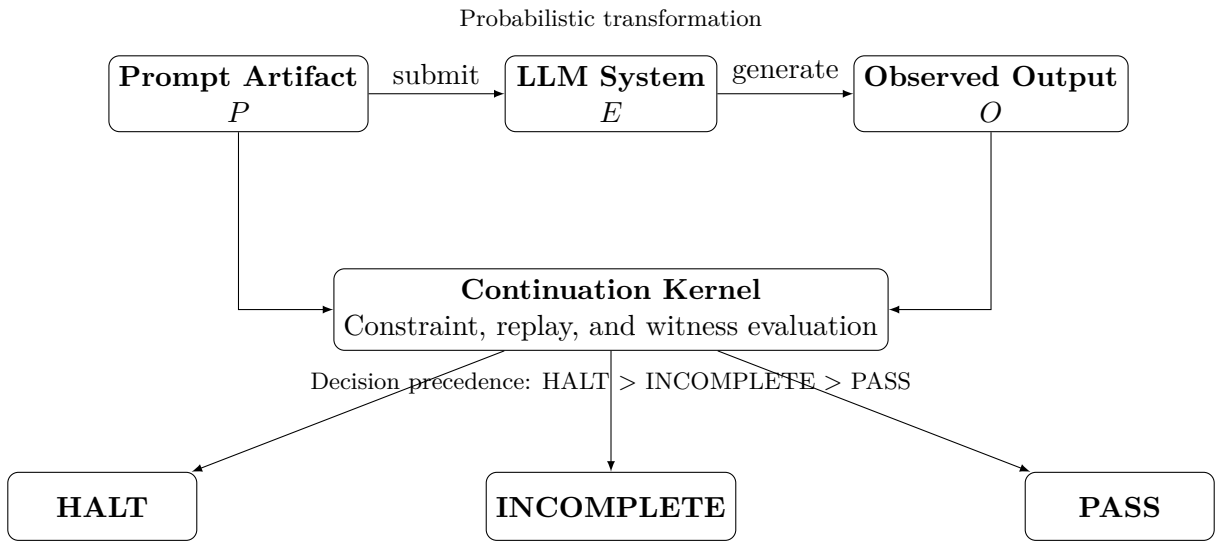

Figure 1: Continuation-control evaluation pipeline for cross-system language admissibility. A prompt artifact P is submitted to an evaluated system E, producing observed output O. The prompt and output are then evaluated by the Continuation Kernel, which determines whether continuation is admissible under ordered precedence HALT > INCOMPLETE > PASS.

6.9 Extension

Future work includes:

- kernel-wrapped LLM execution
- multi-run variance analysis
- automated witness classification

6.10 Summary

Neither evaluated system preserves structural admissibility. Both fail deterministic replay and operator continuity requirements.

6.11 Formal Specification

Let P denote the prompt, E the system, and t the declared execution state. Outputs are

$$O_i = E(P, t)_i$$

Deterministic replay requires

$$\forall i, j,\ \delta(O_i, O_j) = 0$$

Replay failure is defined by

$$\exists i, j :\ \delta(O_i, O_j) > \epsilon$$

with

$$\epsilon = 0$$

Replay failure alone is sufficient to exclude the DIRECT witness class. Let $C(P, O)$ denote preservation of declared constraint structure. Then witness classification is defined as

$$\text{DIRECT} \iff [\forall i, j,\ \delta(O_i, O_j) = 0] \wedge C(P, O) = 1$$

$$\text{WRAPPED} \iff [\exists i, j,\ \delta(O_i, O_j) > 0] \wedge C(P, O) = 1$$

$$\text{DRIFT} \iff [\exists i, j,\ \delta(O_i, O_j) > 0] \wedge C(P, O) = 0$$

6.12 The `run_bundle` Artifact

Attribute	**Metric / Value**
Experiment ID	CC-LA-2026-04
Control Model	HALT > INCOMPLETE > PASS
System A (Claude)	`WRAPPED` ($\Delta > 0$)
System B (Gemini)	`DRIFT` ($\Delta \gg 0$)
Admissibility	**REJECTED**

Table 2: Formal witness classification results for the cross-system language admissibility experiment.

A minimal `run_bundle` for this experiment should include at least the following fields:

```
{
  "experiment_id": "CC-LA-2026-04",
  "timestamp_utc": "...",
  "decision_model": "HALT > INCOMPLETE > PASS",
  "prompt_sha256": "...",
  "system_name": "...",
  "system_version": "...",
  "run_index": 1,
  "output_sha256": "...",
  "witness_class": "...",
  "decision": "...",
  "refusal_record": "... or null"
}
```

The purpose of the `run_bundle` is not descriptive convenience but replay accountability. Without a preserved bundle, output claims from the evaluated systems cannot be promoted to admissible evidence.

6.13 Refusal-First Execution Control

In the Bookbinder framework, failure of operator continuity prohibits permissive continuation. Let G_4 denote the Operator Continuity gate. If G_4 fails, or returns a partial state insufficient to establish one-to-one preservation between input instruction and output action, the system must emit a refusal record and reduce the decision according to ordered precedence

$$\text{HALT} > \text{INCOMPLETE} > \text{PASS}$$

The governing rule is therefore

$$G_4 \in \{\text{FAIL}, \text{PARTIAL}, \text{WEAK}\} \Longrightarrow \text{Refusal Record Required}$$

Operationally, the reason is straightforward: if the mapping from prompt to output cannot be verified as structurally preserved, then the process admits out-of-band transformation and loses replay authority. In such cases the correct system action is not explanation but refusal.

Formally,

$$\neg\text{VerifyOperatorContinuity}(P, O) \Longrightarrow \text{Decision} \in \{\text{HALT}, \text{INCOMPLETE}\}$$

with HALT taking precedence whenever the failure is sufficient to invalidate admissible continuation within the declared execution scope.

A Witness Kernel Reference Implementation

This appendix provides a minimal reference implementation for artifact sealing, manifest verification, and refusal-first continuation checks.

A.1 Oracle Drop Shell Harness

```
#!/usr/bin/env bash
set -euo pipefail

ARTIFACT="${1:-paper.pdf}"
SOURCE="${2:-paper.tex}"
RUN_BUNDLE="run_bundle.json"
REFUSAL="refusal_record.json"
MANIFEST="SHA256_MANIFEST.txt"

if [[ ! -f "$ARTIFACT" ]]; then
  echo '{"decision":"HALT","reason":"missing_artifact"}'
  exit 1
fi

if [[ ! -f "$SOURCE" ]]; then
  echo '{"decision":"INCOMPLETE","reason":"missing_source"}'
  exit 0
fi

shasum -a 256 "$ARTIFACT" "$SOURCE" "$RUN_BUNDLE" "$REFUSAL" > "$MANIFEST"
shasum -a 256 "$MANIFEST" > "${MANIFEST}.sha256"
shasum -a 256 "$RUN_BUNDLE" > "${RUN_BUNDLE}.sha256"
shasum -a 256 "$REFUSAL" > "${REFUSAL}.sha256"

echo '{"decision":"PASS","scope":"artifact_seal","reason":"artifact_present_hash_bound_manifest
```

A.2 Python Verifier

```
#!/usr/bin/env python3
import hashlib
import json
from pathlib import Path
import sys

ROOT = Path(".")

def sha256_file(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(65536), b""):
            h.update(chunk)
    return h.hexdigest()

def fail(reason: str, code: int = 1):
    print(json.dumps({
        "verification": "HALT",
        "reason": reason,
        "promotion": "none"
    }, indent=2))
    sys.exit(code)

def main():
    decision_path = ROOT / "decision.json"
    manifest_path = ROOT / "SHA256_MANIFEST.txt"

    if not decision_path.exists():
        fail("missing_decision_json")

    if not manifest_path.exists():
        fail("missing_manifest")

    decision = json.loads(decision_path.read_text())

    manifest_targets = set()
    for line in manifest_path.read_text().splitlines():
        line = line.strip()
        if not line:
            continue
        parts = line.split()
        if len(parts) < 2:
            fail("manifest_malformed")
        claimed_hash = parts[0]
        claimed_target = Path(parts[-1]).name
        target_path = ROOT / claimed_target
```

```
        if not target_path.exists():
            fail(f"manifest_missing_target:{claimed_target}")
        live_hash = sha256_file(target_path)
        if claimed_hash != live_hash:
            fail(f"manifest_hash_mismatch:{claimed_target}")
        manifest_targets.add(claimed_target)

    required_manifest_targets = {"paper.pdf", "paper.tex", "run_bundle.json", "refusal_record.j
    for needed in required_manifest_targets:
        if needed not in manifest_targets:
            fail(f"manifest_missing_target:{needed}")

    result = {
        "verification": "PASS",
        "scope": "artifact_seal_only",
        "promotion": "none",
        "formal_classification": "SEALED_NOT_REPLAY_CERTIFIED",
        "verified_checks": decision.get("verified_checks", []),
        "not_verified": decision.get("not_verified", [])
    }
    print(json.dumps(result, indent=2))

if __name__ == "__main__":
    main()
```

A.3 Reference Decision Logic

The reference kernel separates artifact sealing from replay certification:

1. If the artifact is missing, return HALT.
2. If required supporting evidence is missing, return INCOMPLETE.
3. If artifacts are present and hash-bound, return PASS at artifact-seal scope only.
4. Replay certification must be established separately and may not be inferred from artifact sealing alone.

B Conclusion

The Babylon Continuation framework establishes a deterministic, artifact-first method for governing execution in language-driven systems. By enforcing replay-verifiable evidence and refusal-first control, the system ensures that continuation is granted only when structurally admissible within the declared evaluation scope. Deterministic replay, cross-host reproduction, and execution equivalence remain unverified conditions in the current execution instance.

References

[1] Anthropic. Claude, 2026. Large language model system referenced in the cross-system admissibility experiment.

[2] J. M. Bookbinder. The babylon continuation: Deterministic replay and refusal-first execution in language-driven systems, March 2026. Working paper, Witness Grade Analytics (WGA), Atlanta, Georgia, USA.

[3] Google. Gemini, 2026. Large language model system referenced in the cross-system admissibility experiment.

[4] National Institute of Standards and Technology. Secure hash standard (SHS). *Federal Information Processing Standards Publication 180-4*, 2015. U.S. Department of Commerce.